Assignment 2 (Weeks 3 and 4)

PART 1

1. The query returns 0 because in SQL, NULL does not evaluate to TRUE — it evaluates to UNKNOWN which doesn't represent a value

The correct query

```
SELECT COUNT(*)
FROM Student
WHERE phone_number IS NULL;
```

In aggregate functions (SUM, AVG, MIN, MAX, COUNT(column)), NULL values are ignored by definition because including an "unknown" value in arithmetic would make the result meaningless

2. The DISTINCT keyword is redundant. GROUP BY department already aggregates the table into unique groups for each department, ensuring the department appears only once per row in the output.

3. The LEFT JOIN will return all customers (including those with 0 orders), while INNER JOIN only returns those with at least one order. The difference tells you that some customers have no orders at all. Specifically, the extra rows returned by LEFT JOIN correspond to customers whose customer_id appears in Customer but has no matching row in Order. INNER JOIN silently excludes these customers; LEFT JOIN preserves them, filling Order columns with NULL.

Scenario causing the discrepancy: A customer registers on the platform (creating a row in Customer) but never places an order (no corresponding row in Order). LEFT JOIN returns this customer with NULL in all Order columns; INNER JOIN omits them entirely. The magnitude of the difference equals the number of such "orderless" customers.

4. The given query is invalid because employee_name is neither grouped nor aggregated. SQL does not know which employee's name should be displayed for each department since multiple employees belong to one department.

Corrected query :

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department; (If avg. department salary is needed)
```

Conceptually: If allowed, the engine would have to arbitrarily pick an employee name from the group, which is logically inconsistent with the aggregation.

5.SQL has a defined logical execution order:
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

It is invalid because WHERE runs *before* groups are formed, so aggregate values do not yet exist at that point you cannot filter on something that hasn't been computed yet. HAVING runs *after* GROUP BY, where aggregate values are available.

Corrected query :

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000 ;
```

6. a) Non-correlated subquery is better as the overall average is calculated once and reused.

```
SELECT * FROM Product

WHERE price > (SELECT AVG(price) FROM Product);
```

b) Correlated subquery is better since the average must be recalculated for each employee's specific department, meaning the inner query must reference the outer row's department value to know which subset to average over.

A correlated subquery executes once for each row in the outer query, making it slower on large datasets. A non-correlated subquery executes only once and is generally more efficient.

PART 2

A.

```
SELECT c.customer_id, c.name
FROM Customer as c
LEFT JOIN order as o
ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL ;
```

A `LEFT JOIN` keeps every customer. Customers without matching orders receive NULL values, allowing us to identify those who never ordered.

B.

```
SELECT restaurant_id, AVG(order_value) AS avg_order_value
FROM(
SELECT o.order_id,
        o.restaurant_id,
        SUM(m.price * oi.quantity) AS order_value
   FROM Order o
   JOIN OrderItem oi ON o.order_id = oi.order_id
   JOIN MenuItem m ON oi.item_id = m.item_id
   GROUP BY o.order_id, o.restaurant_id)
AS order_totals
GROUP BY restaurant_id
HAVING COUNT(*) > 5;
```

The inner query calculates each order's value. The outer query computes the average for each restaurant. `HAVING` filters restaurants with more than five orders.

C.

```
FROM (
    SELECT o.order_id, o.restaurant_id, SUM(m.price * oi.quantity) AS order_value
    FROM Orders o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem m ON oi.item_id = m.item_id
    GROUP BY o.order_id, o.restaurant_id
) AS OrderTotals
GROUP BY restaurant_id
HAVING AVG(order_value) > (
    SELECT AVG(order_value)
```

```
    FROM (
        SELECT o.order_id, SUM(m.price * oi.quantity) AS order_value
        FROM Orders o
        JOIN OrderItem oi ON o.order_id = oi.order_id
        JOIN MenuItem m ON oi.item_id = m.item_id
        GROUP BY o.order_id
    ) AS PlatformOrders
);
```

A non-correlated subquery is sufficient because the platform average is computed once and compared against every restaurant.

D.

```
SELECT restaurant_id,
       item_name,
       total_quantity
FROM
(
    SELECT m.restaurant_id,
           m.item_name,
           SUM(oi.quantity) AS total_quantity,
           RANK() OVER
           (
              PARTITION BY m.restaurant_id
              ORDER BY SUM(oi.quantity) DESC
           ) AS rnk
    FROM MenuItem m
    JOIN OrderItem oi
      ON m.item_id = oi.item_id
    GROUP BY m.restaurant_id,
             m.item_name
) RankedItems
WHERE rnk = 1;
```

Window functions correctly identify the highest-ranked item in every restaurant. Using only GROUP BY and MAX() finds the maximum quantity but does not identify which menu item produced that maximum. It also struggles when multiple items tie for first place.

PART 3

1. The functional dependencies are :
   student_id → student_name
   course_id → course_name,instructor_id
   instructor_id → instructor_name, instructor_office
   { student_id, course_id } → grade, enrollment_date

2. The table is in **1NF** because:

- every attribute contains a single atomic value,
- there are no repeating groups,
- each row is uniquely identified by `(student_id, course_id)`.

3. There are partial dependencies like :
student_id → student_name
course_id → course_name, instructor_id
This violates 2NF because non-key attributes depend on only one part of the primary key

Update anomaly: Course name is repeated in every enrollment row. Change it in one place, forget another → same course now has two different names in the DB.

Insertion anomaly: Can't add a new course to the DB until some student actually enrolls in it. No enrollment row = no place to store the course.

Deletion anomaly: If the last student drops a course and that row gets deleted, the course's details (name, instructor) vanish from the DB entirely.

4. The transitive dependency :
   - course_id → instructor_id → instructor_name, instructor_office
   - course_id → instructor_name, instructor_office

If an instructor changes office, every course taught by that instructor must be updated. Missing one update causes inconsistent office information.

5. Student (<u>student_id,</u> student_name)

Reason: student_id → student_name (Partial dependency separated out)

Course (<u>course_id</u>, course_name, instructor_id)
Reason: course_id → course_name, instructor_id

Instructor (instructor_id, Instructor_name, instructor_office)
Reason: instructor_id → Instructor_name, instructor_office

Enrollment (student_id, course_id, grade, enrollment_date)
Reason: (student_id, course_id) → Grade, enrollment_date

PART 4

For Part II(a), I chose a LEFT JOIN instead of a subquery to identify customers who have never placed an order. The same result could have been achieved using a NOT EXISTS or NOT IN subquery, but the LEFT JOIN combined with WHERE order_id IS NULL is straightforward and easy to understand. It clearly shows that all customers are retained first and only those without matching orders are selected. It also avoids potential issues that NOT IN can have when NULL values exist.

Normalizing the Enrollment table into 3NF removes redundancy and prevents update, insertion, and deletion anomalies by storing students, courses, instructors, and enrollments separately. However, retrieving complete enrollment information now requires multiple joins, making queries more complex and sometimes slower. In real-world systems, limited denormalization is sometimes introduced to improve reporting performance and reduce the cost of frequent joins, especially in analytical workloads.